

any_view

Document #: P3411R1
Date: 2024-12-29
Project: Programming Language C++
Audience: SG9, LEWG
Reply-to: Hui Xie
<hui.xie1990@gmail.com>
Louis Dionne
<ldionne.2@gmail.com>
S. Levent Yilmaz
<levent.yilmaz@gmail.com>

Contents

- 1 Revision History
 - 1.1 R1
 - 1.2 R0
- 2 Abstract
- 3 Motivation
- 4 Design Space and Prior Art
 - 4.1 Boost.Range `boost::ranges::any_range`
 - 4.2 range-v3 `ranges::views::any_view`
- 5 Proposed Design
- 6 Alternative Design for Template Parameters
 - 6.1 Alternative Design 1: Variadic Named Template Parameters
 - 6.2 Alternative Design 2: Single Template Parameter: `RangeTraits`
 - 6.3 Alternative Design 3: Barry's Named Template Argument Approach
- 7 Other Design Considerations
 - 7.1 Why don't follow range-v3's design where first template parameter is `range_reference_t`?
 - 7.2 Name of the `any_view_options`
 - 7.3 `constexpr` Support
 - 7.4 Move-only view Support

- 7.5 Move-only iterator Support
- 7.6 Is `any_view_options::contiguous` Needed ?
- 7.7 Is `any_view` const-iterable?
- 7.8 `common_range` support
- 7.9 `borrowed_range` support
- 7.10 Valueless state of `any_view`
- 7.11 ABI Stability
- 7.12 Performance
- 8 Implementation Experience
- 9 References

§ 1 Revision History

§ 1.1 R1

- Performance tests update
 - All benchmarks use -O3 (as requested in Wrocław)
 - For eager algorithms, do not use ranges at all (as requested in Wrocław)
 - Add a new benchmark test case where `vector` vs `any_view` used in function arguments
- Template parameters redesign
 - `any_view<Foo>` and `any_view<const Foo>` should just work
 - 4 Different alternative designs

§ 1.2 R0

- Initial revision.

§ 2 Abstract

This paper proposes a new type-erased view: `std::ranges::any_view`. That type-erased view allows customizing the traversal category of the view, its value type and a few other properties. For example:

```
class MyClass {
    std::unordered_map<Key, Widget> widgets_;
public:
    std::ranges::any_view<Widget> getWidgets();
};
```

```
std::ranges::any_view<Widget> MyClass::getWidgets() {
    return widgets_ | std::views::values
                   | std::views::filter(myFilter);
}
```

§ 3 Motivation

Since being merged into C++20, the Ranges library has gained an increasingly rich and expressive set of views. For example,

```
// in MyClass.hpp
class MyClass {
    std::unordered_map<Key, Widget> widgets_;
public:
    auto getWidgets() {
        return widgets_ | std::views::values
                       | std::views::filter([](const auto&){ /*...*/ });
    }
};
```

While such use of ranges is exceedingly convenient, it has the drawback of leaking implementation details into the interface. In this example, the return type of the function essentially bakes the implementation of the function into the interface.

In large applications, such liberal use of `std::ranges` can lead to increased header dependencies and often a significant compilation time penalty.

Attempts to separate the implementation into its own translation unit, as is a common practice for non-templated code, are futile in this situation. The return type of the above definition of `getWidgets` is:

```
std::ranges::filter_view<
    std::ranges::elements_view<
        std::ranges::ref_view<std::unordered_map<Key, Widget>>,
        1>,
    MyClass::getWidgets()::<lambda(const auto:11&)>> >
```

While this type is already difficult to spell once, it is much harder and more brittle to maintain it as the implementation or the business logic evolves. These challenges for templated interfaces are hardly unique to ranges: the numerous string types in the language and lambdas are some common examples that lead to similar challenges.

Type-erasure is a very popular technique to hide the concrete type of an object behind a common interface, allowing polymorphic use of objects of any type that model a given concept. In fact, it is a technique commonly employed by the standard. `std::string_view`, `std::function` and `std::function_ref`, and `std::any` are the type-erased facilities for the examples above.

`std::span<T>` is another type-erasure utility recently added to the standard; and is closely related to ranges in fact, by allowing type-erased *reference* of any underlying *contiguous*

range of objects.

In this paper, we propose to extend the standard library with `std::ranges::any_view`, which provides a convenient and generalized type-erasure facility to hold any object of any type that satisfies the `ranges::view` concept itself. `std::ranges::any_view` also allows further refinement via customizable constraints on its traversal categories and other range characteristics.

§ 4 Design Space and Prior Art

Designing a type like `any_view` raises a lot of questions.

Let's take `std::function` as an example. At first, its interface seems extremely simple: it provides `operator()` and users only need to configure the return type and argument types of the function. In reality, `std::function` makes many other decisions for the user:

- Are `std::function` and the callable it contains copyable?
- Does `std::function` own the callable it contains?
- Does `std::function` propagate const-ness?

After answering all these questions we ended up with several types:

- `function`
- `move_only_function`
- `function_ref`
- `copyable_function`

The design space of `any_view` is a lot more complex than that:

- Is it an `input_range`, `forward_range`, `bidirectional_range`, `random_access_range`, or a `contiguous_range`?
- Is the range copyable?
- Is it a `sized_range`?
- Is it a `borrowed_range`?
- Is it a `common_range`?
- What is the `range_reference_t`?
- What is the `range_value_t`?
- What is the `range_rvalue_reference_t`?
- What is the `range_difference_t`?
- Is the range const-iterable?

- Is the iterator copyable for `input_iterator`?
- Is the iterator equality comparable for `input_iterator`?
- Do the iterator and sentinel types satisfy `sized_sentinel_for<S, I>?`

We can easily get a combinatorial explosion of types if we follow the same approach we did for `std::function`. Fortunately, there is prior art to help us guide the design.

§ 4.1 Boost.Range `boost::ranges::any_range`

The type declaration is:

```
template<
    class Value
  , class Traversal
  , class Reference
  , class Difference
  , class Buffer = any_iterator_default_buffer
>
class any_range;
```

Users will need to provide `range_reference_t`, `range_value_t` and `range_difference_t`. `Traversal` is equivalent to `iterator_concept`, which decides the traversal category of the range. Users don't need to specify `copyable`, `borrowed_range` and `common_range`, because all Boost.Range ranges are copyable, `borrowed_range` and `common_range`. `sized_range` and `range_rvalue_reference_t` are not considered in the design.

§ 4.2 range-v3 `ranges::views::any_view`

The type declaration is:

```
enum class category
{
    none = 0,
    input = 1,
    forward = 3,
    bidirectional = 7,
    random_access = 15,
    mask = random_access,
    sized = 16,
};

template<typename Ref, category Cat = category::input>
struct any_view;
```

Here `Cat` handles both the traversal category and `sized_range`. `Ref` is the `range_reference_t`. It does not allow users to configure the `range_value_t`, `range_difference_t`, `borrowed_range` and `common_range`. `copyable` is mandatory in range-v3.

§ 5 Proposed Design

This paper proposes the following interface:

```
enum class any_view_options
{
    input = 1,
    forward = 3,
    bidirectional = 7,
    random_access = 15,
    contiguous = 31,
    sized = 32,
    borrowed = 64,
    move_only = 128
};

template <class Element,
          any_view_options Opts = any_view_options::input,
          class Ref = Element&,
          class Value = remove_cvref_t<Element>,
          class RValueRef =
            add_rvalue_reference_t<remove_reference_t<Ref>>,
          class Diff = ptrdiff_t>
class any_view {
    class iterator; // exposition-only
    class sentinel; // exposition-only

    template <class View>
        requires(!std::same_as<View, any_view> && std::ranges::view<View> &&
                 view_options_constraint<View>())
        any_view(View view);

        any_view(const any_view &) requires (!(Opts &
        any_view_options::move_only));

        any_view(any_view &&) = default;

        any_view &operator=(const any_view &) requires (!(Opts &
        any_view_options::move_only));

        any_view &operator=(any_view &&);

        iterator begin();
        sentinel end();

        size_t size() const requires(Opts & any_view_options::sized);
};

template <class Element, any_view_options Opts, class Ref, class Value,
          class RValueRef,
          class Diff>
inline constexpr bool
    enable_borrowed_range<any_view<Value, Opts, Ref, RValueRef, Diff>> =
    Opts & any_view_options::borrowed;
```

The intent is that users can select various desired properties of the `any_view` by bitwise-oring them. For example:

```
using MyView = std::ranges::any_view<Widget,  
    std::ranges::any_view_options::bidirectional |  
    std::ranges::any_view_options::sized>;
```

§ 6 Alternative Design for Template Parameters

In Wrocław meeting, one important point was made: The majority of the use case of `any_view` is to use it as a function parameter in the API boundary.

```
Bar algo(any_view<Foo>);
```

And in most of cases, the implementation of `algo` only iterate over the range once. The design should make it easy to specify an “input_range only” view, and sometimes “read-only” access to the elements (a `const` reference element type). That is,

```
any_view<Foo>; // should be an input_range where the range_reference_t is  
              Foo&  
any_view<const Foo>; // should be an input_range where the  
                    range_reference_t is const Foo&
```

With the proposed design, the above two use cases would work. Even though there are lots of template parameters, we do not expect users to specify them often because the default would work for majority of the use cases.

§ 6.1 Alternative Design 1: Variadic Named Template Parameters

```
namespace any_view_options {  
  
template <class> struct iterator_concept;  
template <class> struct reference_type;  
template <class> struct value_type;  
template <class> struct difference_type;  
template <class> struct rvalue_reference_type;  
template <bool> struct sized;  
template <bool> struct move_only;  
template <bool> struct borrowed;  
  
} // any_view_options  
  
template <class Element, class... Options>  
class any_view;
```

With this design, the two main use cases would still work

```
using MyView1 = any_view<Foo>; // should be an input_range where the
    range_reference_t is Foo&
using MyView2 = any_view<const Foo>; // should be an input_range where the
    range_reference_t is const Foo&
```

If the default options do not work, users can specify the options in this way:

```
using namespace std::ranges::any_view_options;
using MyView3 = std::ranges::any_view<Foo,

    iterator_concept<std::contiguous_iterator_tag>,
                                reference_type<Foo>,
                                sized<true>,
                                borrowed<true>>>;
```

The benefits of this approach are

- Each parameter is named
- Users do not need to specify the template parameters in a specific order
- Users can skip few template parameters if they need to customize the “last” template option

An implementation of this approach would look like this: [link](#)

However, we believe that this overcomplicates the design.

§ 6.2 Alternative Design 2: Single Template Parameter: RangeTraits

In Wrocław meeting, one feedback we had was to explore the options to have “single expansion point”, i.e not to have too many template parameters

```
struct default_range_traits {};

template <class Element, class RangeTraits = default_range_traits>
class any_view;
```

In the RangeTraits, the user can define aliases to customize iterator_concept, reference_type etc, and define static constexpr bool variables to customize sized, move_only etc. If an alias or static constexpr bool variable is missing, the default type or value will be used.

With this design, the two main use cases would still work

```
using MyView1 = any_view<Foo>; // should be an input_range where the
    range_reference_t is Foo&
using MyView2 = any_view<const Foo>; // should be an input_range where the
    range_reference_t is const Foo&
```

If the default options do not work, users can specify the options in this way:


```

struct MyTraits {
    using iterator_concept = std::contiguous_iterator_tag;
    using reference = int;
    static constexpr bool sized = true;
    static constexpr bool move_only = true;
};

using MyView3 = any_view<int, MyTraits>;

```

The benefits of this approach are

- Each option is named
- Users do not need to specify the template parameters in a specific order
- Users can skip any options if they can use the default values

An implementation of this approach would look like this: [link](#)

However, every time an user needs to customize anything, they need to define a `struct`, which is verbose and inconvenient.

§ 6.2.1 Optional add-on to RangeTraits

If we decided to go with this alternative, we could have a utility that deduces the traits from another range.

```

template <class Range>
struct range_traits {
    using iterator_concept = /* see-below */;
    using reference_type = range_reference_t<Range>;
    using value_type = range_value_t<Range>;
    using rvalue_reference_type = range_rvalue_reference_t<Range>;
    using difference_type = range_difference_t<Range>;

    static constexpr bool sized = sized_range<Range>;
    static constexpr bool move_only = !copyable<decay_t<Range>>;
    static constexpr bool borrowed = borrowed_range<Range>;
};

// MyView4 is a contiguous, sized, copyable, non-borrowed int& range
using MyView4 = any_view<int, range_traits<std::vector<int>>>;

// MyView5 is a contiguous, sized, copyable, non-borrowed const int& range
using MyView5 = any_view<const int, range_traits<const std::vector<int>>>;

```

An implementation of this approach would look like this: [link](#)

§ 6.3 Alternative Design 3: Barry's Named Template Argument Approach

```

template <typename T>
struct type_t {
    using type = T;
};

```

```

};

template <typename T>
inline constexpr type_t<T> type{};

template <class Ref,
         class IterConcept = input_iterator_tag,
         class Value = decay_t<Ref>,
         class RValueRef = remove_reference_t<Ref>&&,
         class Difference = ptrdiff_t>
struct any_view_options {
    type_t<Ref> reference_type;
    type_t<IterConcept> iterator_concept = {};
    bool sized = false;
    bool move_only = false;
    bool borrowed = false;
    type_t<Value> value_type;
    type_t<RValueRef> rvalue_reference_type;
    type_t<Difference> difference_type;
};

template <class Element, any_view_options options = {.reference_type =
    type<Element&>>>
class any_view;

```

This is inspired by Barry's [blog post](#). Thanks to designated initializers and generated CTAD, the user code is extremely readable

```

using MyView1 = any_view<Foo>; // should be an input_range where the
    range_reference_t is Foo&
using MyView2 = any_view<const Foo>; // should be an input_range where the
    range_reference_t is const Foo&

using MyView3 = any_view<int, {.reference_type = type<int&>,
                               .iterator_concept =
                                   type<std::contiguous_iterator_tag>,
                               .value_type = type<long>>>;

using MyView4 = any_view<int, {.reference_type = type<int&>,
                               .iterator_concept =
                                   type<std::contiguous_iterator_tag>,
                               .sized = true,
                               .borrowed = true,
                               .value_type = type<long>>>;

```

Each option is named and user can skip parameters if they want to use the default. However, the user has to follow the same order of the options that are defined in `any_view_options`.

An implementation of this approach would look like this: [link](#)

§ 7 Other Design Considerations

§ 7.1 Why don't follow range-v3's design where first template parameter is `range_reference_t`?

If the first template parameter is `Ref`, we have:

```
template <class Ref,  
          any_view_options Opts = any_view_options::input,  
          class Value = remove_cvref_t<Ref>>
```

For a range with a reference to `T`, one would write

```
any_view<T&>
```

And for a `const` reference to `T`, one would write

```
any_view<const T&>
```

However, it is possible that the user uses `any_view<string>` without realizing that they specified the reference type and they now make a copy of the string every time when the iterator is dereferenced.

§ 7.2 Name of the `any_view_options`

range-v3 uses the name `category` for the category enumeration type. However, the authors believe that the name `std::ranges::category` is too general and it should be reserved for more general purpose utility in ranges library. Therefore, the authors recommend a more specific name: `any_view_options`.

§ 7.3 `constexpr` Support

We do not require `constexpr` in order to allow efficient implementations using e.g. SBO. There is no way, with the current working draft, to construct an object of different type on a `unsigned char[N]` or `std::byte[N]` buffer in `constexpr` context.

§ 7.4 Move-only view Support

Move-only view is worth supporting as we generally support them in ranges. We propose to have a configuration template parameter `any_view_options::move_only` to make the `any_view` conditionally move-only. This removes the need to have another type `move_only_any_view` as we did for `move_only_function`.

We also propose that by default, `any_view` is copyable and to make it move-only, the user needs to explicitly provide this template parameter `any_view_options::move_only`.

§ 7.5 Move-only iterator Support

In this proposal, `any_view::iterator` is an exposition-only type. It is not worth making this iterator configurable. If the iterator is only `input_iterator`, we can also make it

a move-only iterator. There is no need to make it copyable. Existing algorithms that take “input only” iterators already know that they cannot copy them.

§ 7.6 `any_view_options::contiguous` Needed ?

`contiguous_range` is still useful to support even though we have already `std::span`. But `span` is non-owning and `any_view` owns the underlying view.

§ 7.7 `any_view` const-iterable?

We cannot make `any_view` unconditionally const-iterable. If we did, views with cache-on-begin, like `filter_view` and `drop_while_view` could no longer be put into an `any_view`.

One option would be to make `any_view` conditionally const-iterable, via a configuration template parameter. However, this would make the whole interface much more complicated, as each configuration template parameter would need to be duplicated. Indeed, associated types like `Ref` and `RValueRef` can be different between `const` and non-`const` iterators.

For simplicity, the authors propose to make `any_view` unconditionally non-const-iterable.

§ 7.8 `common_range` support

Unconditionally making `any_view` a `common_range` is not an option. This would exclude most of the Standard Library views. Adding a configuration template parameter to make `any_view` conditionally `common_range` is overkill. After all, if users need `common_range`, they can use `my_any_view | views::common`. Furthermore, supporting this turns out to add substantial complexity in the implementation. The authors believe that adding `common_range` support is not worth the added complexity.

§ 7.9 `borrowed_range` support

Having support for `borrowed_range` is simple enough:

- 1. Add a template configuration parameter
- 2. Specialise the `enable_borrowed_range` if the template parameter is set to `true`

Therefore, we recommend conditional support for `borrowed_range`. However, since `borrowed_range` is not a very useful concept in general, this design point is open for discussion.

§ 7.10 Valueless state of `any_view`

We propose providing the strong exception safety guarantee in the following operations: swap, copy-assignment, move-assignment and move-construction. This means that if the operation fails, the two `any_view` objects will be in their original states. If the underlying

view’s move constructor (or move-assignment operator) is not `noexcept`, the only way to achieve the strong exception safety guarantee is to avoid calling these operations altogether, which requires `any_view` to hold its underlying object on the heap so it can implement these operations by swapping pointers. This means that any implementation of `any_view` will have an empty state, and a “moved-from” `any_view` will be in that state.

§ 7.11 ABI Stability

As a type intended to exist at ABI boundaries, ensuring the ABI stability of `any_view` is extremely important. However, since almost any change to the API of `any_view` will require a modification to the vtable, this makes `any_view` somewhat fragile to incremental evolution. This drawback is shared by all C++ types that live at ABI boundaries, and while we don’t think this impacts the livelihood of `any_view`, this evolutionary challenge should be kept in mind by WG21.

§ 7.12 Performance

One of the major concerns of using type erasure is the performance cost of indirect function calls and their impact on the ability for the compiler to perform optimizations. With `any_view`, every iteration will have three indirect function calls:

```
++it;  
it != last;  
*it;
```

While it may at first seem prohibitive, it is useful to remember the context in which `any_view` is used and what the alternatives to it are.

The following benchmarks were compiled with clang 20 with libc++, with `-O3`, run on APPLE M4 MAX CPU with 16 cores. We have done the same benchmarks on a 8 core Intel CPU and they have very similar results.

§ 7.12.1 A naive micro benchmark: iteration over vector vs any_view

Naively, one would be tempted to benchmark the cost of iterating over a `std::vector` and to compare it with the cost of iterating over `any_view`. For example, the following code:

```
std::vector v = std::views::iota(0, state.range(0)) |  
std::ranges::to<std::vector>();  
for (auto _ : state) {  
    for (auto i : v) {  
        benchmark::DoNotOptimize(i);  
    }  
}
```

vs

```
std::vector v = std::views::iota(0, state.range(0)) |  
std::ranges::to<std::vector>();  
std::ranges::any_view<int&> av(std::views::all(v));
```

```

for (auto _ : state) {
    for (auto i : av) {
        benchmark::DoNotOptimize(i);
    }
}

```

Benchmark	Time	Time vector
Time any_view		

[BM_vector vs. BM_AnyView]/1024 1975	+7.3364	237
[BM_vector vs. BM_AnyView]/2048 4042	+7.7186	464
[BM_vector vs. BM_AnyView]/4096 8011	+7.7098	920
[BM_vector vs. BM_AnyView]/8192 15790	+7.6034	1835
[BM_vector vs. BM_AnyView]/16384 31966	+7.7470	3654
[BM_vector vs. BM_AnyView]/32768 66796	+8.1498	7300
[BM_vector vs. BM_AnyView]/65536 132599	+8.0808	14602
[BM_vector vs. BM_AnyView]/131072 263523	+8.0281	29189
[BM_vector vs. BM_AnyView]/262144 495899	+7.4773	58497
OVERALL_GEOMEAN 0	+8.6630	0

We can see that `any_view` is 8.6 times slower on iteration than `std::vector`. However, this benchmark is not a realistic use case. No one would create a `vector`, immediately create a type erased wrapper `any_view` that wraps it and then iterate through it. Similarly, no one would create a `lambda`, immediately create a `std::function` and then call it.

§ 7.12.2 A slightly more realistic benchmark: A view pipeline vs `any_view`

Since `any_view` is intended to be used at an ABI boundary, a more realistic benchmark would separate the creation of the view in a different TU. Furthermore, most uses of `any_view` are expected to be with non-trivial view pipelines, not with e.g. a raw `std::vector`. As the pipeline increases in complexity, the relative cost of using `any_view` becomes smaller and smaller.

Let's consider the following example, where we create a moderately complex pipeline and pass it across an ABI boundary either with a `any_view` or with the pipeline's actual type:

```

// header file
struct Widget {
    std::string name;
    int size;
};

struct UI {
    std::vector<Widget> widgets_;
}

```

```

std::ranges::transform_view<complicated...> getWidgetNames();
};

// cpp file
std::ranges::transform_view<complicated...> UI::getWidgetNames() {
    return widgets_ | std::views::filter([](const Widget& widget) {
        return widget.size > 10;
    }) |
        std::views::transform(&Widget::name);
}

```

And this is what we are going to measure:

```

lib::UI ui = {...};
for (auto _ : state) {
    for (auto& name : ui.getWidgetNames()) {
        benchmark::DoNotOptimize(name);
    }
}

```

In the `any_view` case, we simply replace `std::ranges::transform_view<complicated...>` by `std::ranges::any_view` and we measure the same thing.

Benchmark	Time	Time
complicated	any_view	

[BM_RawPipeline vs. BM_AnyViewPipeline]/1024	1315	2438
		+0.8536
[BM_RawPipeline vs. BM_AnyViewPipeline]/2048	2713	4928
		+0.8162
[BM_RawPipeline vs. BM_AnyViewPipeline]/4096	5637	9570
		+0.6976
[BM_RawPipeline vs. BM_AnyViewPipeline]/8192	11539	19795
		+0.7154
[BM_RawPipeline vs. BM_AnyViewPipeline]/16384	23475	38994
		+0.6611
[BM_RawPipeline vs. BM_AnyViewPipeline]/32768	47792	78278
		+0.6379
[BM_RawPipeline vs. BM_AnyViewPipeline]/65536	96976	156851
		+0.6174
[BM_RawPipeline vs. BM_AnyViewPipeline]/131072	197407	317560
		+0.6087
[BM_RawPipeline vs. BM_AnyViewPipeline]/262144	399623	634694
		+0.5882
OVERALL_GEOMEAN	0	0
		+0.6862

This benchmark shows that `any_view` is about 68% slower on iteration, which is much better than the previous naive benchmark. However, note that this benchmark is still not very realistic. Writing down the type of the view pipeline causes an implementation detail (how the pipeline is implemented) to leak into the API and the ABI of this class, and increases header dependencies, which defeats the purpose of hiding implementation into a separate translation unit.

As a result, most people would instead copy the results of the pipeline into a container and return that from `getWidgetNames()`, for example as a `std::vector<std::string>`. This leads us to our last benchmark, which we believe is much more representative of what people would do in the wild.

§ **7.12.3 A realistic benchmark: A copy of `vector<string>` vs `any_view`**

As mentioned above, various concerns that are not related to performance like readability or API/ABI stability mean that the previous benchmarks are not really representative of what happens in the real world. Instead, people in the wild tend to use owning containers like `std::vector` as a type erasure tool for lack of a better tool. Such code would look like this:

```
// header file
struct UI {
    std::vector<Widget> widgets_;
    std::vector<std::string> getWidgetNames() const;
};

// cpp file
std::vector<std::string> UI::getWidgetNames() const {
    std::vector<std::string> results;
    for (const Widget& widget : widgets_) {
        if (widget.size > 10) {
            results.push_back(widget.name);
        }
    }
    return results;
}
```

Benchmark		Time
Time vector	Time any_view	

[BM_VectorCopy vs. BM_AnyViewPipeline]/1024		-0.7042
8243	2438	
[BM_VectorCopy vs. BM_AnyViewPipeline]/2048		-0.7226
17764	4928	
[BM_VectorCopy vs. BM_AnyViewPipeline]/4096		-0.7379
36516	9570	
[BM_VectorCopy vs. BM_AnyViewPipeline]/8192		-0.7927
95467	19795	
[BM_VectorCopy vs. BM_AnyViewPipeline]/16384		-0.7893
185104	38994	
[BM_VectorCopy vs. BM_AnyViewPipeline]/32768		-0.7890
371035	78278	
[BM_VectorCopy vs. BM_AnyViewPipeline]/65536		-0.8079
816621	156851	
[BM_VectorCopy vs. BM_AnyViewPipeline]/131072		-0.8121
1690305	317560	
[BM_VectorCopy vs. BM_AnyViewPipeline]/262144		-0.8228
3581632	634694	
OVERALL_GEOMEAN		-0.7723
0	0	

With this more realistic use case, we can see that `any_view` is 3 times faster. For the returning vector case, we have some variations of the implementations to produce the vector, including reserve the maximum possible size, or use the range pipelines with `ranges::to`. They all have extremely similar results. In our benchmark, 10% of the Widgets were filtered out by the filter pipeline and the name string's length is randomly 0-30. So some of strings are in the SBO and some are allocated on the heap. We maintain that this code pattern is very common in the wild: making the code simple and clean at the cost of copying data, even though most of the callers don't actually need a copy of the data at all.

In conclusion, we believe that while it's easy to craft benchmarks that make `any_view` look bad performance-wise, in reality this type can often lead to better performance by sidestepping the need to own the data being iterated over.

Furthermore, by putting this type in the Standard library, we would make it possible for implementers to use optimizations like selective devirtualization of some common operations like `for_each` to achieve large performance gains in specific cases.

§ 7.12.4 Another Common Use Case: Function Arguments `vector` vs `any_view`

Another very common use case is that library authors provide an API that takes a range of elements. The library authors would like to hide implementation details in its own library to reduce the header dependencies and avoid leaking implementation details. Due to the lack of type erasure utilities, typically the API takes a `vector`, even though the implementation only needs to iterate once over the elements.

This is the benchmark we are measuring.

```
// algo.hpp
int algo1(const std::vector<std::string>& strings);
int algo2(std::ranges::any_view<std::string> strings);

// algo.cpp
int algo1(const std::vector<std::string>& strings) {
    int result = 0;
    for (const auto& str : strings) {
        if (str.size() > 6) {
            result += str.size();
        }
    }
    return result;
}

int algo2(std::ranges::any_view<std::string> strings)
{
    int result = 0;
    for (const auto& str : strings) {
        if (str.size() > 6) {
            result += str.size();
        }
    }
}
```

```

    }
    return result;
}

```

With the vector version, the user needs to create a temporary vector if they do not have it at the first place. So in our benchmark, we are measuring

```

for (auto _ : state) {
    std::vector<std::string> widget_names;
    widget_names.reserve(ui.widgets_.size());
    for (const auto& widget : ui.widgets_) {
        widget_names.push_back(widget.name);
    }
    auto res = lib::algo1(widget_names);
    benchmark::DoNotOptimize(res);
}

```

With the any_view, we can simply pass in a transform_view

```

for (auto _ : state) {
    auto res = lib::algo2(ui.widgets_ |
        std::views::transform(&Widget::name));
    benchmark::DoNotOptimize(res);
}

```

And here is the result:

Benchmark	Time	Time
vector	Time	any_view

[BM_algo_vector vs. BM_algo_AnyView]/1024	9615	1829
[BM_algo_vector vs. BM_algo_AnyView]/2048	20651	3782
[BM_algo_vector vs. BM_algo_AnyView]/4096	41035	7436
[BM_algo_vector vs. BM_algo_AnyView]/8192	87227	15044
[BM_algo_vector vs. BM_algo_AnyView]/16384	182922	30818
[BM_algo_vector vs. BM_algo_AnyView]/32768	381383	60771
[BM_algo_vector vs. BM_algo_AnyView]/65536	793920	125004
[BM_algo_vector vs. BM_algo_AnyView]/131072	1733654	232982
[BM_algo_vector vs. BM_algo_AnyView]/262144	3592842	488714
OVERALL_GEOMEAN	0	0
		-0.8098
		-0.8169
		-0.8188
		-0.8275
		-0.8315
		-0.8407
		-0.8425
		-0.8656
		-0.8640
		-0.8364

We can see the any_view version is 4 times faster. This is a very common pattern in the real world code. vector has been used in API boundaries as a type-erasure tool.

§ 8 Implementation Experience

`any_view` has been implemented in [range-v3], with equivalent semantics as proposed here. The authors also implemented a version that directly follows the proposed wording below without any issue [ours].

§ 9 References

[ours] Hui Xie, S. Levent Yilmaz, and Dionne Louis. A proof-of-concept implementation of `any_view`.

https://github.com/huixie90/cpp_papers/tree/main/impl/any_view

[range-v3] Eric Niebler. range-v3 library.

<https://github.com/ericniebler/range-v3>